13

Apache Kafka từ số 0

topic · partition · consumer group — giải thích dễ hiểu

Hôm nay mình nói gì

1. Vì sao cần streaming
2. Kafka là một cái log
3. Topic & partition · Offset & consumer group
4. Replication
5. Kafka vs RabbitMQ vs Kinesis

Batch mỗi đêm không đủ khi

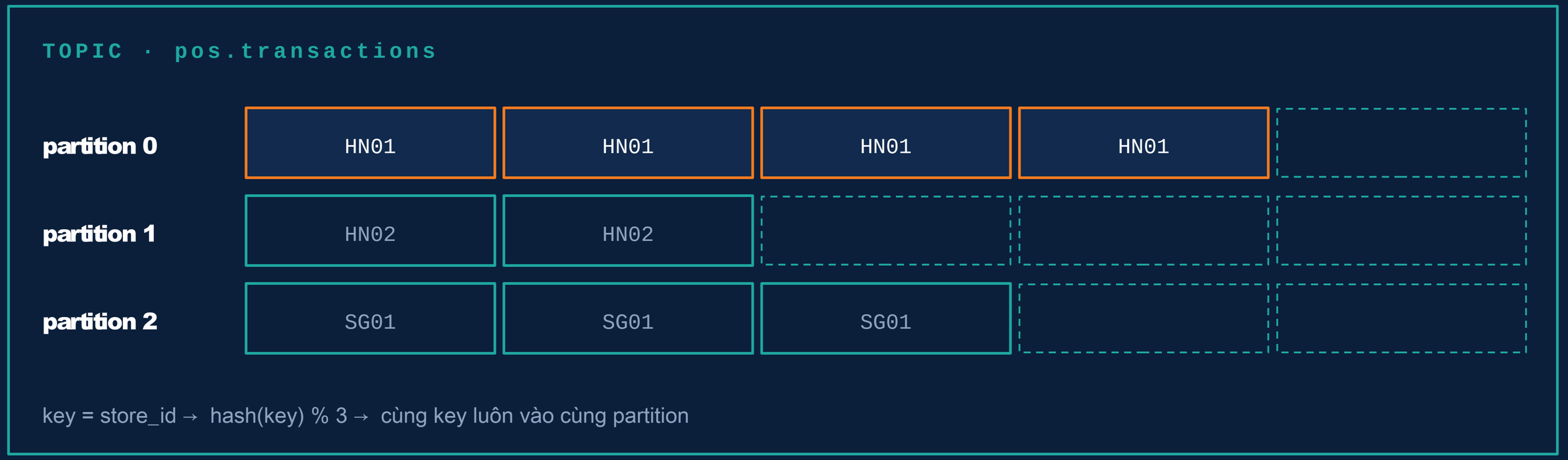
- Máy pha báo lỗi, cần biết ngay
- Tồn kho cập nhật theo đơn
- Gian lận cần chặn trong giây
- Nhiều hệ thống cùng cần một sự kiện

Kafka là một cái log append-only



Ghi vào cuối, đọc từ bất kỳ đâu, không xoá khi đọc.

Topic và partition

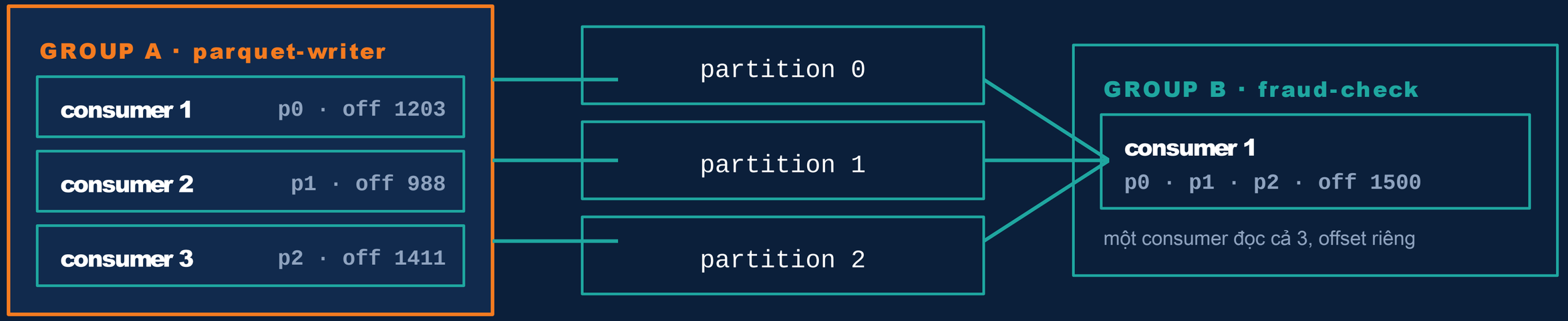


Thứ tự chỉ được đảm bảo trong một partition.

Chọn key và số partition

- Key = cái cần giữ thứ tự: store_id, machine_id
- Số partition = mức song song tối đa của consumer
- Tăng partition được, giảm không
- Key lệch → partition lệch (skew lại xuất hiện)

Offset và consumer group

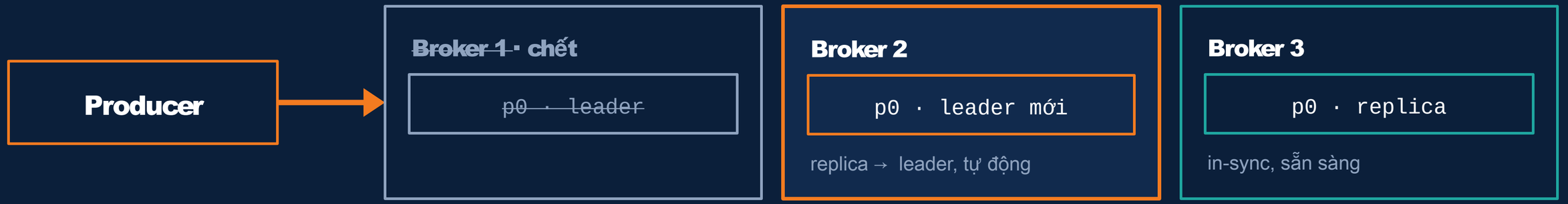


Hai group đọc độc lập cùng dữ liệu.

Consumer group hoạt động

- Thêm consumer → rebalance chia lại partition
- Nhiều consumer hơn partition → có consumer ngồi không
- Commit offset = đánh dấu đã xử lý đến đâu
- Consumer chết → consumer khác trong group nhận partition

Replication



replication.factor = 3 · mỗi partition có 3 bản trên 3 broker

acks = all · producer chờ đủ replica in-sync mới coi là ghi xong

Replication factor 3 là chuẩn production.

Producer

```
from confluent_kafka import Producer
p = Producer({"bootstrap.servers": "kafka:9092", "acks": "all"})

def on_delivery(err, msg):
    if err: log.error("delivery failed: %s", err); return
    log.info("partition=%s offset=%s", msg.partition(), msg.offset())
p.produce(
    topic="pos.transactions",
    key=tx["store_id"],
    value=json.dumps(tx),
    on_delivery=on_delivery,
)
p.flush()
```

Không có key → round-robin, mất thứ tự.

Consumer

```
c = Consumer({
    "bootstrap.servers": "kafka:9092",
    "group.id": "parquet-writer",
    "auto.offset.reset": "earliest",
    "enable.auto.commit": False,
})
c.subscribe(["pos.transactions"])

while True:
    msgs = c.consume(num_messages=500, timeout=1.0)
    if not msgs: continue
    write_parquet([json.loads(m.value()) for m in msgs])
    c.commit(asynchronous=False)   # sau khi ghi xong
```

Commit sau khi xử lý xong, không phải trước.

Kafka vs RabbitMQ vs Kinesis

	Kafka	RabbitMQ	Kinesis
Mô hình	Log	Queue	Log, managed
Giữ message sau đọc	Có, theo retention	Không, xoá khi ack	Có, 24h – 365d
Replay	Có, reset offset	Không	Có
Vận hành	Tự vận hành (hoặc managed)	Tự vận hành	AWS lo
Khi nào chọn	Streaming data, nhiều consumer	Task queue giữa service	Đã ở AWS, team nhỏ

Lỗi hay gặp

SAI	ĐÚNG
~~1 partition "cho đơn giản"~~	Ít nhất bằng số consumer dự kiến
~~Commit offset trước khi xử lý~~	Commit sau
~~Message 5 MB~~	Tham chiếu đến object trên S3
~~Không đặt retention~~	Theo nhu cầu replay

Lab #13 — IoT máy pha → Kafka → Parquet

1. `make up PROFILE=streaming`, mở Kafka UI
2. `coffeegen stream --topic iot.machine` giả lập 60 máy
3. Viết consumer ghi Parquet theo phút lên bronze
4. Chạy 2 consumer cùng group, xem partition chia
5. Tắt 1 consumer, xem rebalance

KẾT QUẢ MONG ĐỢI

Kafka UI: lag về 0

File Parquet xuất hiện mỗi phút

CÂU HỎI PHỎNG VẤN #13

Vì sao Kafka scale được nhờ partition?

Partition là đơn vị song song cho cả ghi, lưu và đọc.

VIDEO TIẾP THEO

14 Exactly-once, at-least-once: đảm bảo dữ liệu không mất, không trùng

Thứ Sáu 18/12 · 19:00

Đăng ký